PROJECT REPORT

NEWS ARTICLE CLASSIFICATION USING NATURAL LANGUAGE PROCESSING AND MACHINE LEARNING

Submitted by

Jyothirmai Chapala - 231FA23076

B.Tech – Computer Science and Engineering (Data Science)
2023–2027

Vignan's Foundation for Science, Technology & Research
(Deemed to be University)
Andhra Pradesh, India

Academic Year: 2026–2027

Abstract

News articles are published in large volumes across digital platforms, making it difficult for readers and content-management systems to manually organize them. This project develops a machine-learning-based system that automatically classifies a news article into one of five categories: business, entertainment, politics, sport, or tech. The workflow includes exploratory data analysis, text cleaning, stop-word removal, stemming, TF-IDF feature extraction, model training, evaluation, and deployment through a Streamlit web interface.

The dataset contains 2,225 labelled news articles. After preprocessing, TF-IDF is used to represent the text numerically with a maximum of 5,000 features. The dataset is split into 1,780 training samples and 445 testing samples. Multinomial Naive Bayes, Logistic Regression, and Support Vector Machine (SVM) models are compared. The SVM model is selected as the best-performing model, achieving a reported test accuracy of 98.2%. The trained model is integrated into a Streamlit application where a user can enter an article and receive a predicted category.

Keywords: News classification, Natural Language Processing, TF-IDF, text preprocessing, machine learning, SVM, Streamlit.

Table of Contents

- 1. Introduction
- 2. Problem Statement
- 3. Objectives
- 4. Dataset Description
- 5. Tools and Technologies
- 6. Methodology
- 7. System Architecture
- 8. Implementation Details
- 9. Results and Discussion
- 10. Testing
- 11. Advantages and Limitations
- 12. Future Enhancements
- 13. Conclusion
- 14. References

1. Introduction

The growth of online news platforms has resulted in a continuous increase in the number of digital news articles. Organizing these articles into meaningful topics is useful for news websites, readers, search systems, and recommendation applications. Manual categorization is time-consuming and may not scale well as the volume of content increases.

Text classification is a Natural Language Processing (NLP) task in which a computer model assigns a predefined label to a text document. In this project, supervised machine learning is used to learn patterns from labelled news articles and predict the category of an unseen article.

2. Problem Statement

To design and implement an automated system that accepts the text of a news article and predicts whether it belongs to the business, entertainment, politics, sport, or tech category. The system should preprocess text, convert it into numerical features, train and evaluate suitable classification algorithms, and provide predictions through a simple user interface.

3. Objectives

- Explore the news dataset and understand its category distribution and article lengths.
- Clean and normalize article text using standard NLP preprocessing techniques.
- Convert text into numerical vectors using TF-IDF feature extraction.
- Train and compare Naive Bayes, Logistic Regression, and SVM classifiers.
- Evaluate the models using test accuracy and a confusion matrix.
- Save the selected model and provide an interactive Streamlit prediction interface.

4. Dataset Description

The project dataset contains 2,225 news articles and two columns: category and text. The category column stores the target label, while the text column contains the article content. Five categories are present.

Category	Number of articles
Sport	511
Business	510
Politics	417
Tech	401
Entertainment	386
Total	2,225

Sport and business have the highest number of samples, while entertainment has the lowest. The class distribution is reasonably spread across the five labels, although the counts are not exactly equal.

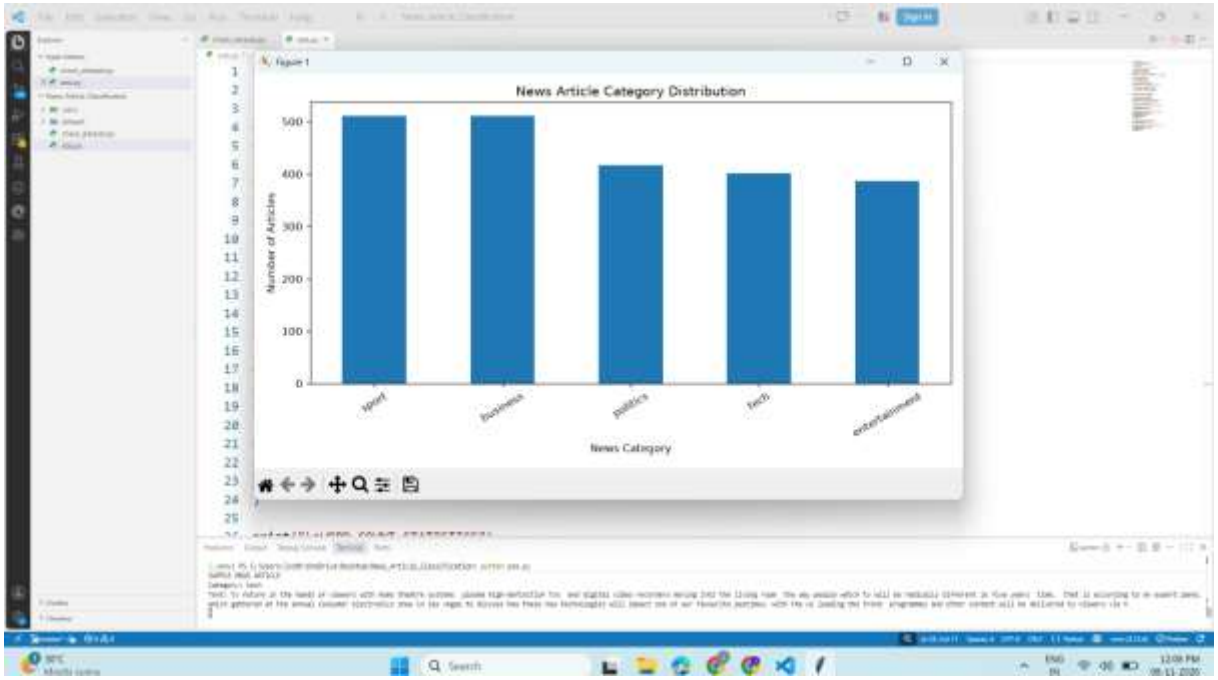


Figure 1. Distribution of news articles across the five categories.

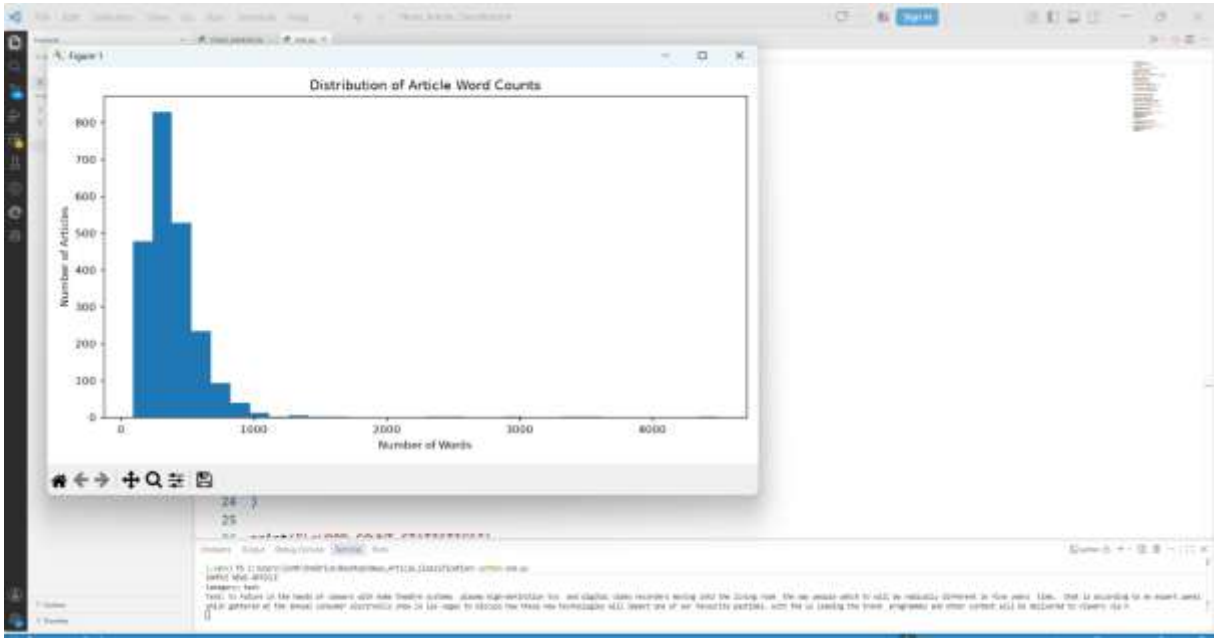


Figure 2. Distribution of article word counts; most articles are concentrated in the lower word-count ranges.

5. Tools and Technologies

Tool / Library	Purpose
Python	Programming language used to implement the workflow.
Pandas and NumPy	Dataset loading, tabular data handling, and numerical operations.
Matplotlib	Visualizing category distribution, word counts, model comparison, and evaluation results.

NLTK	Stop-word removal and Porter stemming for text preprocessing.
Scikit-learn	TF-IDF vectorization, dataset splitting, model training, and evaluation.
Streamlit	Building the interactive web interface for article classification.
VS Code	Development environment used to create and run the Python scripts.

6. Methodology

6.1 Exploratory Data Analysis

The dataset is first loaded and inspected to verify its dimensions, column names, sample records, and category counts. A bar chart is generated to visualize the number of articles in each category. Article word counts are also examined using a histogram to understand the distribution of document lengths.

6.2 Text Preprocessing

Raw news text may contain inconsistent casing, punctuation, common words, and different grammatical forms of the same word. Preprocessing reduces this noise and helps the model focus on informative terms. The implemented preprocessing pipeline includes:

1. Convert text to lowercase.
2. Remove punctuation and unwanted non-alphabetic characters according to the cleaning logic.
3. Tokenize the text into words.
4. Remove common English stop words using the NLTK stop-word list.
5. Apply Porter stemming to reduce words to their stem forms.

For example, words such as “playing”, “played”, and “plays” may be reduced to a common stem. The cleaned dataset is saved for the next stage.

6.3 TF-IDF Feature Extraction

Machine-learning algorithms require numerical input. Term Frequency–Inverse Document Frequency (TF-IDF) converts each article into a vector whose values reflect the importance of terms within a document relative to the complete collection. The implementation uses a maximum of 5,000 features and includes unigrams and bigrams.

The training feature matrix has 1,780 rows and 5,000 columns, while the testing feature matrix has 445 rows and 5,000 columns. Each row represents one article and each column represents a vocabulary feature.

6.4 Dataset Splitting

The dataset is divided into training and testing subsets. The training subset contains 1,780 articles (80%), and the testing subset contains 445 articles (20%). The implementation uses a stratified split with a fixed random state so that the class proportions are maintained as closely as possible and the split can be reproduced.

6.5 Model Training

Three supervised classification algorithms are trained using the TF-IDF feature vectors:

- Multinomial Naive Bayes: a probabilistic classifier commonly used for text classification.

- Logistic Regression: a linear classification algorithm that estimates class probabilities.
- Support Vector Machine (SVM): a classifier that finds decision boundaries separating classes; the project uses a linear SVM approach.

The models are compared using the held-out test set. The SVM model is selected as the best model based on the reported evaluation accuracy.

7. System Architecture

The system follows a sequential NLP and machine-learning pipeline:

News Dataset → Data Exploration → Text Preprocessing → TF-IDF Vectorization → Train/Test Split → Model Training and Comparison → Best Model Saved → Streamlit Interface → Predicted News Category

During prediction, the article entered by the user is passed through the same preprocessing and TF-IDF transformation used during training. The saved classifier then predicts one of the five supported categories. Reusing the fitted vectorizer is essential because the prediction text must be represented using the same feature vocabulary as the training data.

8. Implementation Details

Script / Component	Responsibility
check_dataset.py	Loads the dataset and prints sample rows, shape, column names, and category distribution.
eda.py	Generates visualizations for category counts and article word-count distribution.
preprocess.py	Cleans article text, removes stop words, applies stemming, and saves the cleaned dataset.
tfidf.py	Splits the data and creates TF-IDF feature matrices with 5,000 features.
train.py	Trains and compares the three classifiers, evaluates the selected model, plots results, and saves the best model.
Streamlit application	Accepts article text, applies the saved preprocessing/vectorization/model pipeline, and displays the predicted category.

The training output reports SVM as the best model with a test accuracy of 98.2%, and confirms that the best model was saved successfully. The Streamlit application displays the selected model and test accuracy, accepts article text, and shows the predicted category along with an option to view preprocessed text.

9. Results and Discussion

The project successfully completed dataset inspection, exploratory analysis, text preprocessing, feature extraction, model comparison, evaluation, and application deployment.

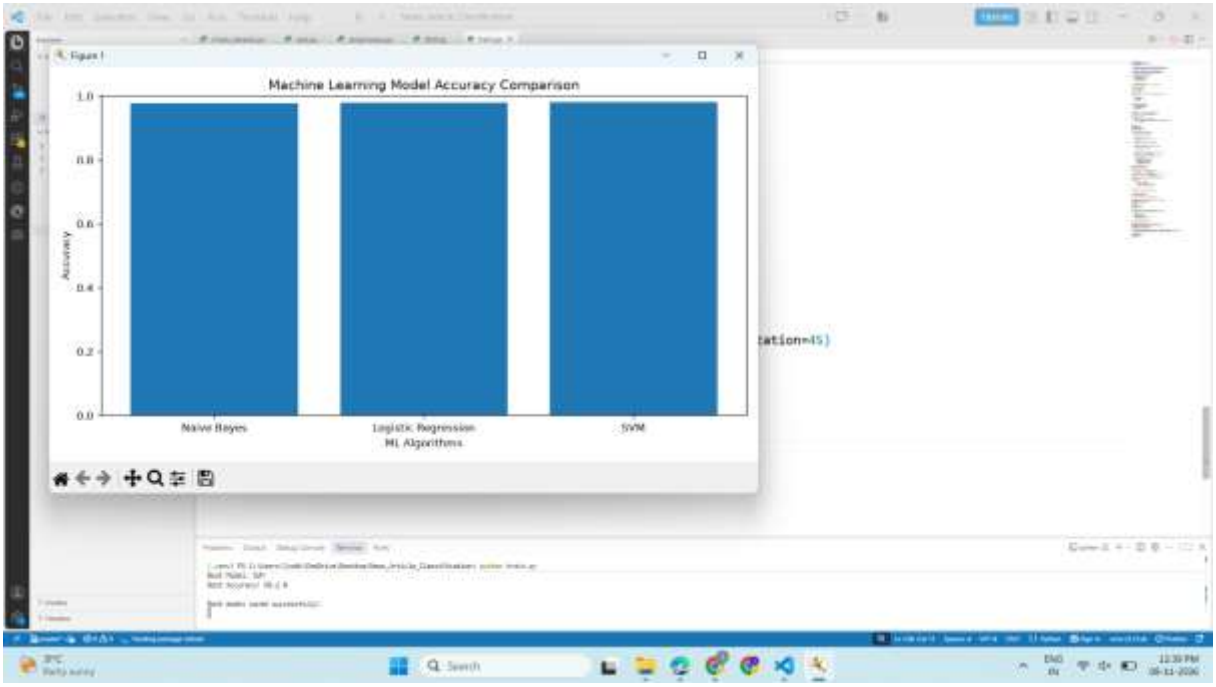


Figure 3. Accuracy comparison of Naive Bayes, Logistic Regression, and SVM.

The supplied comparison chart shows the three model results visually. Exact numerical values for Naive Bayes and Logistic Regression were not available in the terminal output or readable chart labels, so they are intentionally not reported here.

The training terminal output identifies SVM as the best model and reports 98.2% test accuracy. This corresponds to approximately 437 correctly classified articles out of 445, subject to rounding of the reported percentage.

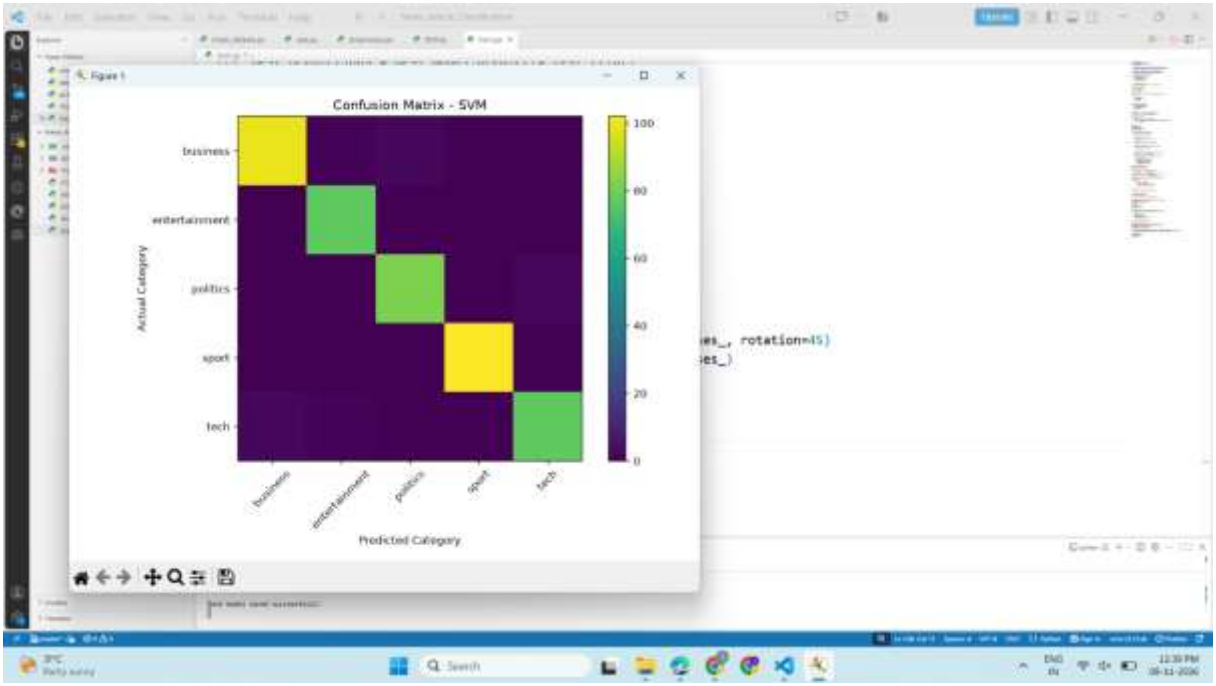


Figure 4. Confusion matrix for the selected SVM classifier.

The confusion matrix shows strong diagonal values, indicating that most test articles were assigned to their correct categories. Off-diagonal cells represent misclassifications between categories. The matrix supports the conclusion that the classifier performs well across the five classes.

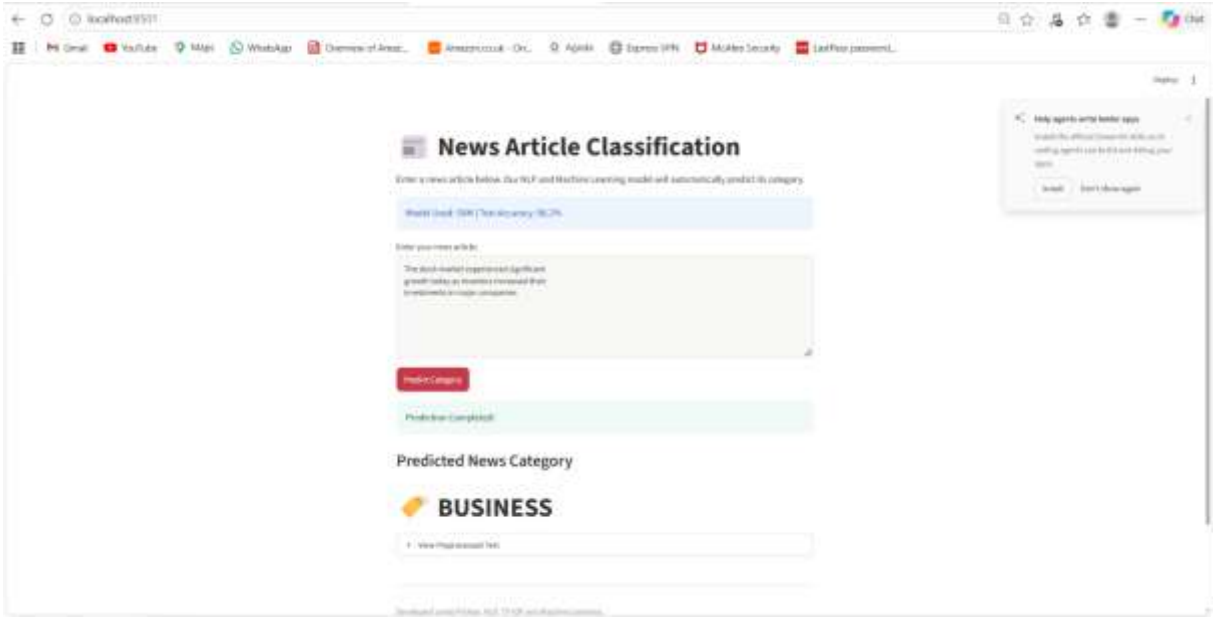


Figure 5. Streamlit application displaying a sample news article prediction.

The application screenshot shows the model information, article input area, prediction button, successful prediction message, and predicted category. In the displayed example, the article is classified as BUSINESS.

10. Testing

The following functional checks can be used to verify the application. The expected result is that the interface returns one of the five supported labels for valid news text.

Test case	Input / Action	Expected result
TC-01	Launch the Streamlit application.	The application page loads without an error.
TC-02	Enter a business-related article and click Predict Category.	A category prediction is displayed.
TC-03	Enter a sports-related article.	The result is one of the supported category labels.
TC-04	Submit an empty text field.	The application should request valid article text or prevent an empty prediction, if this validation is implemented.
TC-05	Expand View Preprocessed Text.	The cleaned version of the entered article is displayed, if the section is selected.

The supplied screenshot confirms a successful prediction for one sample input. For a formal submission, test all five categories and record the actual input text and observed output for each test.

11. Advantages and Limitations

11.1 Advantages

- Automatically categorizes news articles and reduces manual sorting effort.
- Uses a straightforward and reproducible NLP pipeline.
- Compares multiple machine-learning algorithms before selecting a model.
- Provides a simple browser-based interface through Streamlit.
- Achieves a reported test accuracy of 98.2% on the held-out test set.

11.2 Limitations

- The classifier is limited to the five categories represented in the training dataset.
- Performance may decrease on articles from different sources, writing styles, or time periods.
- TF-IDF mainly captures term frequency patterns and does not fully understand context or word meaning.
- The system is designed for English text and may not work reliably for other languages.
- The dataset is relatively small compared with the volume of news available online.

12. Future Enhancements

- Expand the dataset with more recent articles and a wider range of publishers.
- Add more categories and support multilingual news classification.
- Evaluate precision, recall, F1-score, and per-class performance in addition to accuracy.
- Experiment with advanced language representations such as Word2Vec, GloVe, or transformer-based models.
- Improve the interface with prediction confidence, article history, and batch upload.
- Deploy the application to a cloud hosting platform and monitor performance on new data.

13. Conclusion

The News Article Classification project demonstrates how Natural Language Processing and supervised machine learning can be combined to automatically organize textual news content. The system processes 2,225 labelled articles across five categories, cleans the text, extracts up to 5,000 TF-IDF features, and compares Naive Bayes, Logistic Regression, and SVM classifiers. Based on the reported test results, SVM is selected as the best-performing model with 98.2% accuracy. A Streamlit application makes the trained model accessible through a simple text-input interface. The project provides a complete practical workflow from raw text data to an interactive classification application.